均值模糊（Mean Blurring）笔记

图像处理学习者

2026 年 3 月 28 日

目录

1 概述

均值模糊（Mean Blurring），也称为平均滤波或盒式滤波（Box Filter），是图像处理中最基础、最常用的空间域滤波技术之一。它属于线性滤波方法，通过计算像素邻域内所有像素值的平均值来替换中心像素值，从而达到平滑图像、减少噪声的效果。

在数字图像处理中，均值模糊被广泛应用于图像预处理、噪声抑制、细节消除等场景。作为卷积神经网络和其他计算机视觉算法的预处理步骤，均值模糊能够有效地去除高频噪声，为后续处理提供更干净的输入数据。

2 基本概念

定义 2.1 (均值模糊)**.** *均值模糊是一种线性空间滤波操作，它将图像中每个像素的值替换为其邻域内所有像素值的算术平均值。对于图像 I 中的像素 (x, y)，其模糊后的值 $I'(x, y)$ 可表示为：*

$$I'(x, y) = \frac{1}{N} \sum_{(i,j)\in\Omega} I(x + i, y + j)$$

其中 Ω 表示以 (x, y) 为中心的邻域窗口，N 是窗口内的像素总数。

示例 2.1 (3×3 均值滤波)**.** *对于 3×3 的均值滤波核，每个像素的新值是其 3×3 邻域内 9 个像素的平均值。例如，对于像素 $I(x, y)$：*

$$I'(x, y) = \frac{1}{9} \sum_{i=-1}^{1} \sum_{j=-1}^{1} I(x + i, y + j)$$

对应的滤波核（卷积核）为：

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

3 基本原理

均值模糊的本质是卷积操作。在图像处理中，卷积是通过一个称为"核"(kernel)或"掩模"(mask)的小矩阵在图像上滑动，对每个位置的像素及其邻域进行加权求和。

定理 3.1 (均值模糊的卷积表示)**.** *均值模糊可以表示为图像 I 与均值滤波核 K 的离散卷积：*

$$I' = I * K$$

其中 $*$ 表示卷积运算，滤波核 K 的所有元素值相等，且和为 1。

证明. 设图像 I 的大小为 $M \times N$，滤波核 K 的大小为 $m \times n$（通常 m, n 为奇数）。对于输出图像 I' 中的每个像素 (x, y)：

$$I'(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j) \cdot I(x + i, y + j)$$

其中 $a = \lfloor m/2 \rfloor$, $b = \lfloor n/2 \rfloor$。对于均值模糊，$K(i, j) = 1/(m \times n)$ 对所有 i, j 成立。 □

3.1 滤波核的特性

- **归一化**：滤波核的所有元素之和为 1，保证图像平均亮度不变
- **对称性**：均值滤波核是对称的，具有各向同性
- **非负性**：所有元素均为非负值，确保滤波不会产生负像素值

4 使用场景

4.1 图像去噪

均值模糊能有效去除加性高斯白噪声（AWGN）和均匀分布噪声。对于像素值受到独立同分布噪声污染的情况，通过取平均值可以降低噪声方差。

4.2 预处理与降采样

在图像金字塔构建、图像缩放等操作前，常使用均值模糊来防止混叠效应。模糊处理能去除高频成分，避免下采样时产生摩尔纹。

4.3 细节消除

在目标检测、边缘检测等任务中，有时需要先消除不必要的纹理细节，使主要特征更加突出。均值模糊可以平滑细小纹理，保留主要结构。

4.4 艺术效果制作

在图像编辑软件中，均值模糊可用于创建柔光效果、背景虚化等艺术效果，使图像看起来更加柔和。

5 解决的问题

- **噪声抑制**：有效降低随机噪声的影响，提高图像信噪比
- **细节平滑**：消除细小纹理和细节，突出主要特征
- **混叠预防**：在下采样前去除高频成分，防止混叠效应
- **计算简化**：作为更复杂滤波（如高斯模糊）的快速近似

6 优缺点分析

6.1 优点

- **计算简单**：只需加法和除法，实现效率高
- **易于理解**：原理直观，参数少（仅核大小）
- **各向同性**：对各个方向的平滑效果一致
- **保持均值**：不改变图像的整体亮度水平

6.2 缺点

- **边缘模糊**：在平滑噪声的同时也会模糊图像边缘
- **振铃效应**：在处理锐利边缘时可能产生伪影
- **权重均匀**：对邻域内所有像素同等对待，无法区分噪声和信号
- **细节损失**：会丢失重要的图像细节和纹理信息

7 代码示例

7.1 Python + OpenCV 示例

Listing 1: Python 中使用 OpenCV 实现均值模糊

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

def mean_blur_demo():
    # 读取图像
    img = cv2.imread('input.jpg')

    # 转换为灰度图像（可选）
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 添加高斯噪声用于演示去噪效果
    noise = np.random.normal(0, 25, gray.shape).astype(np.uint8)
    noisy_img = cv2.add(gray, noise)

    # 应用不同大小的均值滤波器
    kernel_sizes = [3, 5, 7, 9]
    blurred_images = []

    for ksize in kernel_sizes:
        # 使用均值模糊
        blurred = cv2.blur(noisy_img, (ksize, ksize))
        blurred_images.append(blurred)

        # 也可以使用boxFilter函数
        # blurred = cv2.boxFilter(noisy_img, -1, (ksize, ksize))

    # 显示结果
    plt.figure(figsize=(12, 8))

    plt.subplot(2, 3, 1)
    plt.imshow(gray, cmap='gray')
    plt.title('原始图像')
    plt.axis('off')

```

```python
    plt.subplot(2, 3, 2)
    plt.imshow(noisy_img, cmap='gray')
    plt.title('添加噪声后的图像')
    plt.axis('off')

    for i, (ksize, img_blurred) in enumerate(zip(kernel_sizes, blurred_images)):
        plt.subplot(2, 3, i+3)
        plt.imshow(img_blurred, cmap='gray')
        plt.title(f'{ksize}×{ksize}均值模糊')
        plt.axis('off')

    plt.tight_layout()
    plt.show()

    # 计算PSNR评估去噪效果
    for ksize, img_blurred in zip(kernel_sizes, blurred_images):
        mse = np.mean((gray - img_blurred) ** 2)
        if mse == 0:
            psnr = float('inf')
        else:
            psnr = 20 * np.log10(255.0 / np.sqrt(mse))
        print(f'核大小 {ksize}×{ksize}: PSNR = {psnr:.2f} dB')

if __name__ == "__main__":
    mean_blur_demo()
```

7.2 C++ + OpenCV 示例

Listing 2: C++中使用 OpenCV 实现均值模糊

```cpp
#include <opencv2/opencv.hpp>
#include <iostream>
#include <vector>

using namespace cv;
using namespace std;

void meanBlurDemo() {
    // 读取图像
    Mat img = imread("input.jpg");
    if (img.empty()) {
        cout << "无法读取图像文件" << endl;
        return;
    }

    // 转换为灰度图像
    Mat gray;
    cvtColor(img, gray, COLOR_BGR2GRAY);

```

```
    // 添加高斯噪声
    Mat noisy_img = gray.clone();
    Mat noise = Mat(gray.size(), gray.type());
    randn(noise, 0, 25);
    noisy_img += noise;

    // 不同核大小的均值模糊
    vector<int> kernel_sizes = {3, 5, 7, 9};
    vector<Mat> blurred_images;

    for (int ksize : kernel_sizes) {
        Mat blurred;

        // 方法1: 使用blur函数
        blur(noisy_img, blurred, Size(ksize, ksize));

        // 方法2: 使用boxFilter函数 (等价于blur)
        // boxFilter(noisy_img, blurred, -1, Size(ksize, ksize));

        blurred_images.push_back(blurred);
    }

    // 显示结果
    string window_names[] = {
        "原始图像", "噪声图像",
        "3x3均值模糊", "5x5均值模糊",
        "7x7均值模糊", "9x9均值模糊"
    };

    vector<Mat> display_images = {gray, noisy_img};
    display_images.insert(display_images.end(),
                          blurred_images.begin(), blurred_images.end());

    for (size_t i = 0; i < display_images.size(); i++) {
        namedWindow(window_names[i], WINDOW_NORMAL);
        imshow(window_names[i], display_images[i]);
    }

    waitKey(0);
    destroyAllWindows();

    // 计算并显示PSNR
    for (size_t i = 0; i < kernel_sizes.size(); i++) {
        Mat diff;
        absdiff(gray, blurred_images[i], diff);
        diff.convertTo(diff, CV_32F);
        diff = diff.mul(diff);

        Scalar s = sum(diff);
        double mse = s[0] / (gray.rows * gray.cols);
```

```
        double psnr = 10.0 * log10((255.0 * 255.0) / mse);

        cout << "核大小" << kernel_sizes[i] << "x" << kernel_sizes[i]
             << ": PSNR = " << psnr << " dB" << endl;
    }
}

int main() {
    meanBlurDemo();
    return 0;
}
```

8 数学公式补充

8.1 离散卷积公式

对于二维离散图像 $I[m,n]$ 和滤波核 $K[i,j]$，卷积运算定义为：

$$I'[m,n] = (I * K)[m,n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} K[i,j] \cdot I[m-i, n-j]$$

8.2 均值滤波的频率响应

均值模糊在频域相当于一个低通滤波器。$N \times N$ 均值滤波器的频率响应为：

$$H(\omega_1, \omega_2) = \frac{1}{N^2} \cdot \frac{\sin(\omega_1 N/2)}{\sin(\omega_1/2)} \cdot \frac{\sin(\omega_2 N/2)}{\sin(\omega_2/2)} \cdot e^{-j\omega_1(N-1)/2} \cdot e^{-j\omega_2(N-1)/2}$$

8.3 噪声方差减少

假设图像受到独立同分布、均值为 0、方差为 σ^2 的加性噪声污染。应用 $N \times N$ 均值滤波后，噪声方差减少为：

$$\sigma'^2 = \frac{\sigma^2}{N^2}$$

信噪比（SNR）提高 N^2 倍。

9 变体与改进

9.1 加权均值滤波

为邻域内不同位置的像素分配不同的权重，通常中心像素权重最大：

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

9.2 自适应均值滤波

根据局部图像统计特性（如局部方差）动态调整滤波强度。在平滑区域使用较大核，在边缘区域使用较小核或不滤波。

9.3 快速实现方法

利用积分图像（Integral Image）可以快速计算任意矩形区域的和，从而实现 $O(1)$ 时间复杂度的均值滤波，与核大小无关。

10 总结与展望

均值模糊作为最基础的图像滤波技术，在图像处理中扮演着重要角色。它简单高效，适用于许多预处理和去噪场景。然而，其均匀加权特性导致在边缘保持方面的局限性，这催生了更先进的滤波方法如高斯模糊、双边滤波、非局部均值等。

在实际应用中，选择滤波方法需要权衡去噪效果、边缘保持、计算复杂度等因素。对于实时性要求高、噪声类型简单的情况，均值模糊仍然是一个实用选择。

注记 10.1.

- 均值模糊会使图像边缘变得模糊，这在某些应用中可能不可接受
- 核大小选择需要权衡：大核去噪效果好但细节损失多，小核细节保持好但去噪效果有限
- 对于脉冲噪声（椒盐噪声），中值滤波通常比均值滤波更有效
- 在深度学习时代，基于神经网络的去噪方法在许多任务上超越了传统滤波方法

参考文献

[1] OpenCV Documentation. *Image Filtering*. https://docs.opencv.org/

[2] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.

[3] Sonka, M., Hlavac, V., & Boyle, R. (2014). *Image Processing, Analysis, and Machine Vision* (4th ed.). Cengage Learning.